Theoretische Informatik 1
Woche 2

NEA

Nichtdeterministische **E**ndliche **A**utomaten

… 5. Übergangsfunktion: $\delta: Q \times \Sigma \rightarrow \mathcal{P}(Q)$

$\mathcal{P}(Q)$ = Potenzmenge von Q = die Menge aller Teilmengen von Q
$= \{S \mid S \subseteq Q\}$

▸ Fragen zur Potenzmenge

$\mathcal{P}(\{1, 2, 3\})$ = "die Menge, die alle Teilmengen von {1, 2, 3} enthält, und sonst nichts."

Fragen:

- ist $1 \in \mathcal{P}(\{1, 2, 3\})$? Nein, aber {1}
- ist $\emptyset \in \mathcal{P}(\{1, 2, 3\})$? Ja
- ist $\{\emptyset\} \in \mathcal{P}(\{1, 2, 3\})$? Nein
- ist $\varepsilon \in \mathcal{P}(\{1, 2, 3\})$? Nein
- ist $\{3, 1\} \in \mathcal{P}(\{1, 2, 3\})$? Ja, da {1, 3} = {3, 1}

$\mathcal{P}(\{1, 2, 3\}) = \{\emptyset, \{1\}, \{2\}, \{3\}, \{1, 2\}, \{1, 3\}, \{2, 3\}, \{1, 2, 3\}\}$

weitere Anmerkungen:

- $\mathcal{P}(\emptyset) = \{\emptyset\}$
- $\mathcal{P}(\{\varepsilon\}) = \{\emptyset, \{\varepsilon\}\}$
- $\mathcal{P}(\{\emptyset\}) = \{\emptyset, \{\emptyset\}\}$

$\emptyset \neq \{\emptyset\} \neq \{\{\emptyset\}\} \neq \{\{\{\emptyset\}\}\}$

ε-NEA

… 5. Übergangsfunktion: $\delta: Q \times (\Sigma \cup \{\varepsilon\}) \rightarrow \mathcal{P}(Q)$

Frage: In folgendem ε-NEA ist der einzige direkte Übergang von Zustand x nach Zustand y ein ε-Übergang:

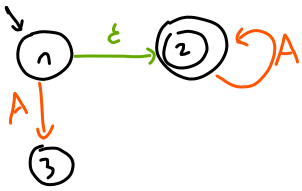


Welches Zeichen liest der Automat, wenn er von x nach y übergeht?
Antwort: gar keins! 😜

es muss $\varepsilon \notin \Sigma$ sein!

▸ **Warum gibt es keinen ε-DEA?**

Weil man, sobald ein ε-Übergang vorliegt, nichtdeterministisch ist.



vom Startzustand erreiche ich nach Lesen eines A entweder den Zustand 2 oder auch den Zustand 3 ⇒ nicht-deterministisch!

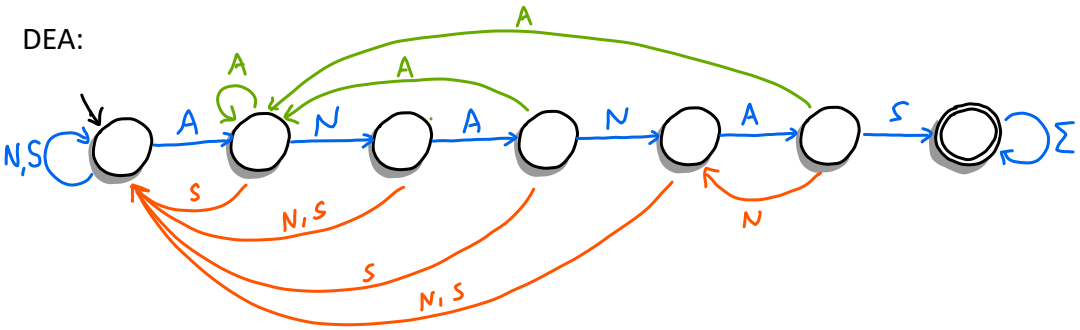
Wozu NEA und ε-NEA?

NEA und ε-NEA können zwar nicht mehr als DEA, aber sie sind machmal einfacher/praktischer hinzuschreiben.

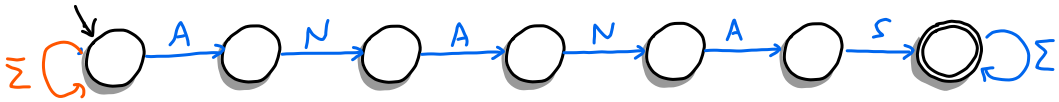
▸ **Beispiel 1:**

L = {w ∈ {A, N, S}* | w enthält den Teilstring "ANANAS"}

DEA:



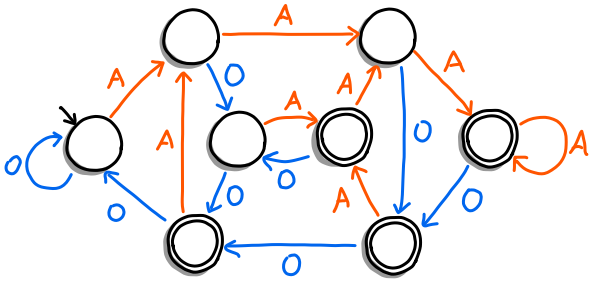
NEA:



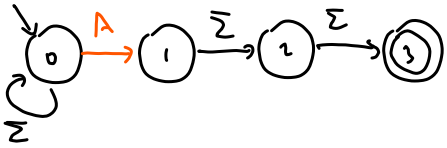
▸ **Beispiel 2:**

L = {w ∈ Σ* | A ist *drittletzter* Buchstabe in w}

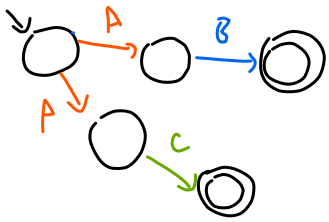
DEA:



NEA:



Anmerkung: Es geht hier nicht nur darum, dass der Automat die Länge des Eingabestrings nicht kennt:



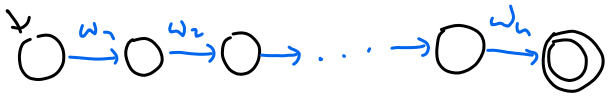
⇦ Im Startzustand muss der Automat entscheiden, ob er zum einen oder zum anderen Zustand wechselt, je nachdem, ob der zweite Buchstabe ein B oder ein C sein wird.

Wie macht der NEA das?

Was bedeutet es, dass ein Automat nichtdeterministisch ist? Was genau tut er, wenn er mehrere Übergansmöglichkeiten hat? Wie kann man sich das vorstellen

▸ 1. Ansatz: "der Mathematiker"

Es ist definiert: Ein NEA *akzeptiert* einen String $w = w_1w_2...w_n \in \Sigma^*$, genau dann, wenn im NEA ein Pfad von s zu einem akzeptierenden Endzustand *existiert*, in dem w vollständig gelesen wird.



Zusätzlich kann es auch andere Pfade durch den NEA eben, die im Startzustand s starten und in denen w vollständig gelesen wird, die aber nicht in einem akzeptierenden Endzustand enden; aber das ist egal! Der String wird dennoch akzeptiert, sobald einer dieser Pfade in einem akzeptierenden Endzustand enden.

Der Mathematiker sagt: das ist alles, was man wissen muss! Eine darüber hinaus gehende "Vorstellung" davon, was der NEA "macht", ist unnötig.

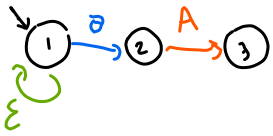
Vorteil: keine Scherereien mit "Vorstellungen"
Nachteil: Leute finden das unintuitiv 😟

▸ 2. Ansatz: "der Fleißige"

»Der Automat probiert nacheinander *alle möglichen Pfade* aus, in dem er w lesen kann und prüft, ob einer davon in einem akzeptierenden Endzustand hält.«

Vorteil: jetzt kann man sich endlich vorstellen, was der Automat tatsächlich tut.
Nachteile:
- Suche braucht evtl. sehr lange.
- Als Laufzeit definiert man später aber nur die *Länge* des erfolgreichen Pfades, nicht aber wie viele Pfade untersucht worden sind.
- Bei ε-NEAs kann es sogar *beliebig lange Pfade* geben, die das Eingabewort einlesen, und darum auch unendlich viele Pfade, die man überprüfen müsste.

Beispiel:



⇦ Akzeptiert der Automat "BA"?

Pfade, in denen "BA" gelesen wird: 1→2→3, 1→1→2→3, 1→1→1→2→3, 1→1→1→1→2→3, ...
⇒ Erst nach Prüfung *unendlich* vieler Pfade durch den Automaten wird festgestellt, dass der DEA den String BA nicht akzeptiert. 😟

▸ 3. Ansatz: "der Multitasker"

»Der Automat probiert alle möglichen Pfade, in denen er w lesen kann, *gleichzeitig* aus, und zwar indem er sich immer die Menge aller Zustände merkt, in denen er gerade sein *könnte*.«

Vorteile:
- Löst die Nachteile vom 2. Ansatz: Das Problem der Laufzeit erübrigt sich.
- Entspricht der Umwandlung von NEA in DEA.

Nachteile:

- Sobald der Automat eine *zusätzlich Datenstruktur* bekommt, geht das schief!
z.B. *Kellerautomat* = Automat, der Daten auf einem *Stapel* ablegen kann.
z.B. *Turingmaschine* = Automat, der auf einem *Speicherband* Buchstaben lesen und schreiben kann.

⇒ verschiedene Programmabläufe kann man nicht mehr einfach parallel ausführen, indem man sich lediglich Zustandsmengen merkt, sondern man müsste sich auch alle möglichen Inhalte der Datenstruktur merken.

▸ 4. Ansatz: "der Wahrsager"

»Der Automat hat die Fähigkeit, den richtigen Pfad *zu raten*.«

Vorteile: Funktioniert auch prima mit Kellerautomaten und Turingmaschinen.
Nachteil: irgendwie seltsam! 😳 😎

▸ Beispiel Zahlen faktorisieren

Gegeben: Zahl n ∈ $\mathbb{N}$, n keine Primzahl
Gesucht: Zahlen a, b ∈ $\mathbb{N}$ mit a ≠ 1 und b ≠ 1 und a · b = n

deterministische Methode: Probiere alle Zahlen a ∈ $\mathbb{N}$ aus mit a > 1 und a ≤ $\sqrt{n}$, ob a die Zahl n teilt. Sobald eine solche Zahl gefunden wurde, ist b = $\frac{n}{a}$.

nichtdeterministische Methode:
1. Rate a und b
2. Beweise, dass a · b = n
3. fertig!

Wichtig hierbei ist, dass dem "Raten" eine Verifikation folgt, dass also bewiesen wird, dass das, was "geraten" wurde, auch korrekt ist.

Weitere Anwendung von Automaten bei der Stringsuche:

Textindex

Index = Datenstruktur zur schnellen Suche
Textindex = Index zur Suche in einem Text

Letzte Vorlesung: $L = \{w \in \{A, N, S\}^* \mid w \text{ enthält den Teilstring "ANANAS"}\}$
jetzt: $L = \{w \in \{A, N, S\}^* \mid w \text{ ist } \textit{Teilstring} \text{ von "ANANAS"}\}$ ⇦ damit kann man schnell in "ANANAS" suchen.

Gesucht ist ein DEA.

▸ **Einschub Definitionen**

Sei $w = w_1w_2...w_n \in \Sigma^*$, $w_i \in \Sigma$ für alle $i \in \{1, 2, ..., n\}$
Dann sei:

- $\{w_1w_2...w_i \mid i \in \{1, 2, ..., n\}\} \cup \{\varepsilon\}$ die Menge der *Präfixe* von w ⇦ die "Anfänge" von w
- $\{w_iw_{i+1}...w_n \mid i \in \{1, 2, ..., n\}\} \cup \{\varepsilon\}$ die Menge der *Suffixe* von w ⇦ die "Enden" von w
- $\{w_iw_{i+1}...w_{j-1}w_j \mid i \in \{1, 2, ..., n\}, j \in \{i, ..., n]\} \cup \{\varepsilon\}$ die Menge der *Teilstrings* von w.

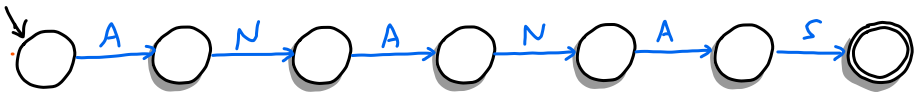
Beobachtung: Jeder Teilstring ist auch der Präfix eines Suffixes.

Beispiel "ANANAS":

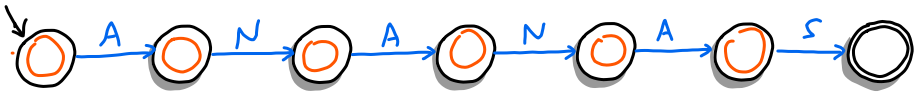
- Präfixe = {ε, A, AN, ANA, ANAN, ANANA, ANANAS}
- Suffixe = {ANANAS, NANAS, ANAS, NAS, AS, S, ε}

Zurück zu der Sprache $L = \{w \in \{A, N, S\}^* \mid w \text{ ist } \textit{Teilstring} \text{ von "ANANAS"}\}$

▸ **1. Schritt:** Akzeptiere nur "ANANAS"



▸ **2. Schritt:** Akzeptiere alle *Präfixe* von "ANANAS"

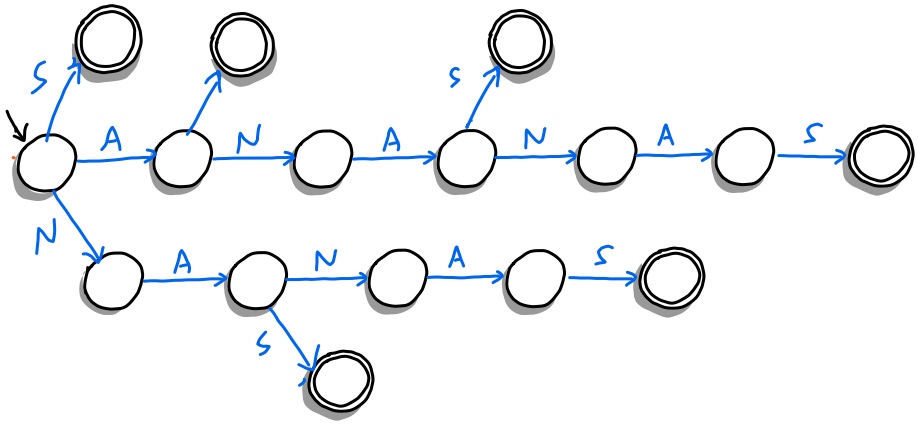


Mache alle Zustände akzeptierend ⇒ alle Präfixe der bisher akzeptierten Strings werden nun auch akzeptiert.

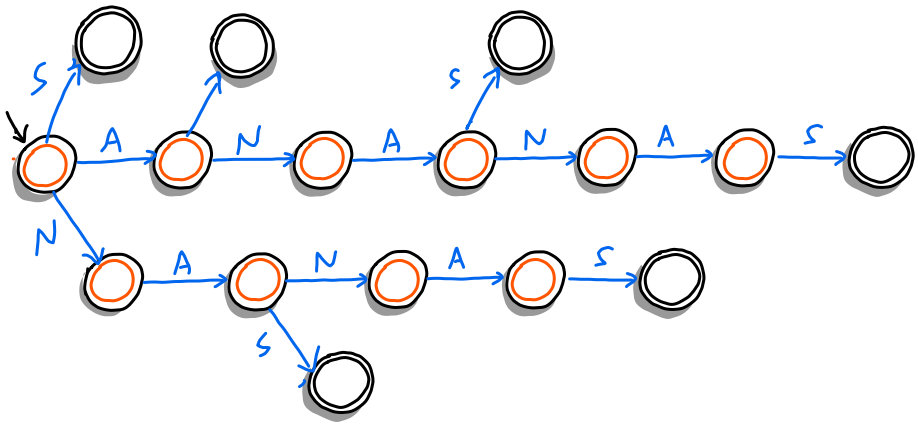
▸ **3. Schritt:** Suffixe von ANANAS akzeptieren:

Anmerkung: Den Automaten aus dem 2. Schritt einfach "umzudrehen" funktioniert nicht. Dann akzeptiert der Automat nämlich die Präfixe des umgedrehten Strings "SANANA"; das sind aber nicht die Suffixe von "ANANAS".

Lösungsidee: Beginne mit dem Automaten aus dem 1. Schritt. Füge nun einen Suffix nach dem anderen ein.
Für jeden Suffix tue folgendes:
1. Laufe vom Startzustand aus durch den Automaten und lese dabei den Suffix so weit es geht.
2. Wenn es nicht mehr weitergeht, dann füge dort einen neuen "Ast" ein.



▸ **4. Schritt:** Teilstrings von ANANAS akzeptieren:
Wende den Trick aus dem 2. Schritt auf den Automaten aus dem 3. Schritt an
⇒ der Automat akzeptiert nun alle Präfixe aller Suffixe von ANANAS, d.h. jeden Teilstring von ANANAS.



Diese Datenstruktur nennt man "*Suffix Trie*".